

CS2103/T Week 3 Lecture Notes

Topics: Git branching, pull requests, build automation, Java production topics, code quality, and developer testing

Source: CS2103/T Week 3 Topics PDF, exported 25 Aug 2026. These notes reorganize the source into lecturer-style explanations for study.

How to Read Week 3

Big picture. Week 3 is about moving from small programming exercises into software engineering practice: collaborate using branches and PRs, automate builds, organize Java code properly, keep code quality high, and test early.

- Git topics prepare you for team work, even if the individual project only uses them for practice.
- Build automation matters because a real product is not just source code; it must be compiled, tested, packaged, and possibly deployed.
- JavaDoc, file I/O, packages, access modifiers, and JAR files are Java production-code tools.
- Coding standards make a codebase readable as one coherent product rather than many personal styles.
- Developer testing and unit testing help catch bugs while the search space is still small.

W3.1 RCS: Branching

Week 3 starts with Git branching. Although these techniques are not strictly needed for a tiny individual project, the course introduces them now so that the same habits are ready for the team project.

What a Branch Is

A branch is a movable line of development. You can work on a feature branch without disturbing the main branch. When the feature is ready, the branch can be merged back.

- **Local branch:** A branch that exists in your local repository.
- **Remote branch:** A branch visible on a remote such as GitHub, often used for sharing work.
- **Keeping branches in sync:** The practice of updating your branch with changes from another branch or remote so your work does not drift too far behind.

Lecturer explanation. Branching is a controlled way to say: I want to experiment or build something,

but I do not want to damage the stable line of work while doing it.

Week item	Branching focus	What to remember
W3.1a	Branching locally.	Create and switch branches locally; keep unrelated work separated.
W3.1b	Keeping branches in sync.	Bring updates across branches so integration surprises are smaller.
W3.1c	Working with remote branches.	Share branches through GitHub and coordinate work beyond your machine.

W3.2 RCS: Creating Pull Requests

A pull request, or PR, is a request to merge one branch into another, normally reviewed on GitHub. The source says you need to create one for your project this week.

A PR is not merely a button on GitHub. It is a communication object: it shows what changed, why the change exists, and gives others a place to review, ask questions, and discuss improvements before merging.

- **Key idea:** Week 3 introduces pull requests as the usual way to propose, review, discuss, and merge changes on GitHub.
- **Practical purpose:** Use PRs to review code before accepting it into a main line of work.
- **Good PR habit:** Keep the change focused; the smaller and clearer the PR, the easier it is to review.

W3.3 Automating the Build Process

Why Build Automation Exists

As a project grows, many repeated steps sit between editing code and delivering an executable product. Build automation tools automate those steps using build scripts.

A non-trivial build can include pulling code, compiling, linking, running automated tests, updating release documents such as build numbers, packaging the program, pushing artifacts to a repository, deploying to a server, cleaning temporary files, and notifying developers.

Build step	Meaning
Compile	Convert source code into a form the machine or runtime can execute.
Link/package	Assemble compiled code and resources into a deliverable form.
Run automated tests	Check whether important behavior still works after changes.
Update release documents	Record build numbers or release metadata automatically.
Deploy	Put the built product where users or servers can run it.
Clean temporary files	Remove intermediate build/test files so future runs start cleanly.

Builds can be run on demand, scheduled, or triggered by events such as a push to revision control. IDEs automate some steps through their build buttons, but larger projects usually use specialized build tools.

- **Java build tools:** Gradle, Maven, Apache Ant, and GNU Make are examples relevant to Java developers.
- **Other ecosystems:** Grunt is associated with JavaScript and Rake with Ruby.
- **Dependency management:** Some build tools also download and manage third-party libraries, which is important because library versions change over time.

Continuous Integration and Continuous Deployment

Continuous integration, or CI, is an extreme use of build automation where integration, building, and testing happen automatically after each code change.

Continuous deployment, or CD, extends CI by also deploying integrated changes to end users automatically.

Concept	Main idea	Examples of tools
CI	Integrate, build, and test automatically after code changes.	Travis, Jenkins, Appveyor, CircleCI, GitHub Actions.
CD	Automatically deploy changes as well as integrate and test them.	Often implemented using the same CI/CD platforms.

W3.4 Java: JavaDoc, File I/O, Packages, Access Modifiers, JARs

JavaDoc: What and Why

JavaDoc is a tool that generates HTML API documentation from specially formatted comments in source code. Modern IDEs also use JavaDoc comments to show explanatory tooltips while you code.

Lecturer explanation. Normal comments help someone reading the source file. JavaDoc comments help both source readers and users of your API who may only see generated documentation or IDE tooltips.

JavaDoc: Method and Class Comments

A method JavaDoc comment should explain what the method returns or does, describe parameters using `@param`, describe return values using `@return`, and document exceptional behavior using `@throws` when relevant.

```
/**
 * Returns the lateral location of the specified position.
 * If the position is unset, NaN is returned.
 *
 * @param x X coordinate of position.
 * @param y Y coordinate of position.
 * @param zone Zone of position.
 * @return Lateral location.
 * @throws IllegalArgumentException If zone is <= 0.
 */
public double computeLocation(double x, double y, int zone) {
    // ...
}
```

A class JavaDoc comment should describe the role represented by the class. For example, a Point class can be documented as representing a location in a 2D space.

File I/O with java.io.File

The java.io.File class represents a file or directory path. A File object can be used to inspect file properties such as absolute path, whether the file exists, and whether it is a directory.

```
File f = new File("data/fruits.txt");
System.out.println(f.getAbsolutePath());
System.out.println(f.exists());
System.out.println(f.isDirectory());
```

Path note. On Windows, a backslash in a string has special meaning, so use "data\\fruits.txt" or simply use forward slash "data/fruits.txt", which also works on Windows.

Reading and Writing Text Files

A Scanner can read from a File object line by line. Because the file might not exist, code that reads a file often handles FileNotFoundException.

```
File f = new File("data/fruits.txt");
Scanner s = new Scanner(f);
while (s.hasNext()) {
    System.out.println(s.nextLine());
}
```

A FileWriter can write text to a file. You must close the FileWriter so the write operation completes properly.

```
FileWriter fw = new FileWriter("temp/lines.txt");
fw.write("first line" + System.lineSeparator() + "second line");
fw.close();
```

To append instead of overwrite, pass true as the second argument to the FileWriter constructor.

```
FileWriter fw = new FileWriter("temp/lines.txt", true);
fw.write("third line");
fw.close();
```

The `java.nio.file.Files` utility class and `java.nio.file.Paths` can perform useful file operations such as copy and delete.

```
Files.copy(Paths.get("data/fruits.txt"), Paths.get("temp/fruits2.txt"));
Files.delete(Paths.get("temp/fruits2.txt"));
```

Packages

Packages organize Java types such as classes, interfaces, and enums. A source file declares its package using a package statement at the very top of the file.

- **First-line rule:** The package statement must be the first line of the source file, and there can be at most one package statement.
- **Folder match:** The package should match the source file's folder path.
- **Compiled output:** The compiler places `.class` files in a folder structure matching the package name.
- **Naming convention:** Package names are lowercase, separated by dots. Java platform packages begin with `java` or `javax`.
- **Company convention:** Companies often start package names with their reversed Internet domain name, such as `com.foo.bar.doohickey.util`.

```
package seedu.tojava.util;

public class Formatter {
    public static final String PREFIX = ">>";
    public static String format(String s) {
        return PREFIX + s;
    }
}
```

Imports and Fully Qualified Names

To use a public member from another package, you can either write its fully qualified name or import the type/package member.

- **Specific import:** `import seedu.tojava.util.StringParser;` imports one class.
- **Wildcard import:** `import seedu.tojava.frontend.*;` imports classes directly inside that package.
- **Fully qualified name:** You can write `seedu.tojava.logic.Processor.getStatus()` without importing `Processor`.
- **Sub-package warning:** Importing a package does not import its sub-packages; packages look hierarchical but imports do not cascade.
- **No package warning:** Leaving a type outside any package is acceptable for small examples but not recommended for real projects.

Static imports allow static members to be used without writing the class name each time.

```
import static seedu.tojava.util.Formatter.PREFIX;
import static seedu.tojava.util.Formatter.format;

String formatted = format("Hello");
boolean isFormatted = formatted.startsWith(PREFIX);
```

When compiling or running packaged Java classes from the command line, the package matters. If `seedu.tojava.Main` is in `Main.java`, compile from the source folder with `javac seedu/tojava/Main.java` and run from the compiler output folder with `java seedu.tojava.Main`.

Access Modifiers

Access modifiers determine which other classes can use a class, field, constructor, or method.

Modifier	Class	Package	Subclass	World
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
no modifier	Yes	Yes	No	No
private	Yes	No	No	No

At class level, `public` makes a class visible everywhere; `no modifier` makes it package-private. At member level, `public`, `protected`, `package-private`, and `private` control how widely a member can be used.

Access levels matter when using external classes and when designing your own classes. For your own code, choose the narrowest access that still allows the code to work cleanly.

JAR Files

Java applications are commonly delivered as JAR files. JAR stands for Java Archive. A JAR can contain Java classes and other resources such as icons or media files.

An executable JAR can be launched with `java -jar`, for example `java -jar foo.jar`. IDEs and build tools such as Gradle can help package an application as a JAR.

W3.5 Code Quality: Coding Standards

Production code needs high quality because modern society depends heavily on software. Poor quality code becomes expensive, risky, and hard to maintain.

Why Coding Standards Matter

A coding standard, also called a style guide, defines how code should look. It covers details such as brace placement, indentation, line length, naming style, and similar conventions.

Central idea. The aim of a coding standard is for the whole codebase to look as if it was written by one person.

- The whole team or company should follow the same standard.
- A project should not invent a style wildly different from normal industry practice unless there is a strong reason.
- Developers should find out the project's coding standard and follow it closely.
- IDEs can enforce some style rules automatically, such as indentation.

AI Impact on Code Quality and Coding Standards

AI changes who applies the standard, but not whether the developer must understand the standard. The agent may handle mechanical style rules, but you still accept or reject the code.

- **Applying rules by hand matters less:** IDEs and AI agents can handle many mechanical details such as indentation, braces, and line length.
- **Consistency matters more:** You read more code than you write, and generated code can arrive in large amounts. A tidy codebase gives better cues to future generated code.
- **Tools cannot judge meaning fully:** A linter can check camelCase, but it cannot always tell whether a name is misleading.
- **Human judgment remains central:** Good names, useful comments, and methods short enough to understand are still code-quality decisions.

W3.6 Developer Testing

Developer testing is testing performed by developers themselves, as opposed to testing done by dedicated testers or end users.

Why Test Early

Delaying testing until the full product is complete has serious disadvantages. The larger the system, the harder it is to locate the cause of a failure.

Problem with late testing	Why it is dangerous
Large search space	A failure may be caused by millions of lines of code or many developers' changes.
Major rework	A bug from requirements or design may require changing much more than a small code section.
Hidden bugs	One bug can hide other bugs that appear only after the first one is fixed.
Delayed delivery	Too many late bugs can force release delays.

Rule of thumb. The earlier a bug is found, the easier and cheaper it is to fix. That is why developer testing is a normal part of serious software work.

W3.7 Unit Testing

Test Drivers

A test driver is code that drives the software under test. It invokes the SUT with test inputs and checks whether the behavior is expected.

The source gives a PayrollTest example: create a Payroll object, set employee data, call totalSalary(), and fail the test if the returned salary differs from the expected value.

```
Payroll p = new Payroll();
p.setEmployees(new String[]{"E001", "E002"});
if (p.totalSalary() != 6400) {
    throw new Error("case 1 failed");
}
```

JUnit and Automated Test Tools

JUnit is a Java tool for automated testing. It provides annotations such as `@Test` and assertion methods such as `assertEquals`, `assertNull`, `assertNotNull`, `assertTrue`, and `assertFalse`.

```
@Test
public void testTotalSalary() {
    Payroll p = new Payroll();
    p.setEmployees(new String[]{"E001", "E002"});
    assertEquals(6400, p.totalSalary());
}
```

Modern IDEs usually integrate with JUnit, so test results can be run and viewed inside the IDE.

What Unit Testing Means

Unit testing means testing individual units, such as methods, classes, or subsystems, to ensure each piece works correctly. In OOP code, it is common to write one or more unit tests for each public method of a class.

- **Test class naming:** For a class `Foo`, the common test class name is `FooTest`.
- **Test method naming:** A useful convention is `unitBeingTested_descriptionOfInputs_expectedOutcome`.
- **Example:** `intDivision_zeroDivisor_exceptionThrown` tells you the method, situation, and expected result.

JUnit Example: IntPair

The source uses an `IntPair` class with fields `first` and `second`, an `intDivision()` method, and a `toString()` method. The `intDivision()` method returns `first / second` using integer division and throws an exception if `second` is 0.

Test method	Purpose
<code>intDivision_nonZeroDivisor_success</code>	Checks normal division, integer truncation, and zero dividend with non-zero divisor.
<code>intDivision_zeroDivisor_exceptionThrown</code>	Checks that division by zero throws the expected exception message.
<code>testStringConversion</code>	Checks that <code>toString()</code> returns text such as 4,7.

The example verifies exceptions using try/catch and fail(). If the expected exception is not thrown, execution reaches fail(), causing the test to fail. If the exception is thrown, the catch block can verify the exception message.

Testing mindset. Write tests that can catch bugs, not only tests that confirm the easiest happy path. Good tests often include tricky inputs that future changes might mishandle.

The source notes that the IntPair examples omit a test for input (0, 0), which should also check the zero-divisor exception path.

Stubs

A proper unit test should test a unit in isolation, so bugs in dependencies do not affect the result. Stubs are one way to isolate the SUT from its dependencies.

A stub has the same interface as the component it replaces, but its implementation is very simple. It mimics responses for a limited set of predetermined inputs, usually by hard-coding those responses.

- **Problem:** A Logic class depends on a Storage class. If the test uses DatabaseStorage, a database bug can make the Logic test fail.
- **Stub solution:** Replace DatabaseStorage with StorageStub during the test.
- **Why this helps:** The test is now focused on Logic instead of being affected by Storage implementation bugs.
- **Limitation:** The stub only knows how to respond to the specific inputs the test needs.

```
class StorageStub implements Storage {
    public String getName(int index) {
        if (index == 5) {
            return "Adam";
        }
        throw new UnsupportedOperationException();
    }
}
```

The source also mentions related kinds of test replacements: mocks, fakes, dummies, and spies. The key Week 3 point is that stubs help test a component in isolation from dependencies.

How Week 3 Connects to Your iP

Week 3 idea	Connection to the project
-------------	---------------------------

Branching	Use branches to isolate work such as adding a new iP level or refactoring parser code.
Pull requests	Open a PR to practice reviewing a focused set of changes before merging.
Build automation	Later, Gradle can compile, test, and package your Java project more reliably than manual commands.
JavaDoc	Use class and method comments to explain public behavior and exceptions.
File I/O	Upcoming iP levels often store tasks in files, so File, Scanner, FileWriter, Files, and Paths become relevant.
Packages	As the codebase grows, move classes into logical packages instead of leaving everything in the default package.
Access modifiers	Use private fields/methods where possible and expose only what other classes really need.
JAR files	Package the chatbot into an executable form users can run with <code>java -jar</code> .
Coding standards	Keep style consistent so future changes and AI-assisted edits remain readable.
Unit testing	Test parsing, task formatting, storage, and command behavior in small units.

Exam and Tutorial Style Questions

- Explain why the course asks you to practice branching in the iP before it becomes necessary in the tP.
- Describe what a pull request is and why it supports code review.

- List several steps that a build automation tool can perform.
- Differentiate CI and CD.
- Explain what JavaDoc generates and why IDEs can benefit from JavaDoc comments.
- Show how File, Scanner, FileWriter, Files, and Paths relate to simple text-file operations.
- Explain why package statements must match folder structure.
- Explain why importing a package does not import its sub-packages.
- Compare public, protected, package-private, and private access.
- Explain what a JAR file is and how an executable JAR can be launched.
- Explain the goal of a coding standard.
- Explain why AI increases the importance of code review and human code-quality judgment.
- Explain why testing late makes debugging more expensive.
- Define test driver, JUnit test, unit test, assertion, and stub.
- Explain why a test using a real dependency may not be a pure unit test.

One-Page Recap

- Branches separate lines of work; remote branches allow collaboration through GitHub.
- Pull requests package changes for discussion and review before merging.
- Build automation turns repeated build, test, package, deploy, clean-up, and notification steps into scripted processes.
- CI automatically integrates, builds, and tests after changes; CD also deploys continuously.
- JavaDoc turns structured source comments into API documentation and IDE tooltips.
- File I/O in Java can use File, Scanner, FileWriter, Files, and Paths for simple text-file work.
- Packages organize Java types; package names should match folder paths and use lowercase dot-separated names.
- Access modifiers control visibility at class and member level.
- JAR files package Java classes and resources; executable JARs can run with `java -jar`.
- Coding standards make code look consistent and easier to maintain.
- Developer testing catches bugs earlier, when they are cheaper to locate and fix.
- Unit testing checks small units in isolation; JUnit provides `@Test` and assertion methods.
- Stubs replace dependencies with simple hard-coded behavior so a unit can be tested in isolation.

Source Coverage Map

- **W3.1a-c:** Branching locally, keeping branches in sync, and working with remote branches.
- **W3.2a:** Creating pull requests.
- **W3.3a-c:** Integration, build automation, dependency management, CI, and CD.

- **W3.4a-b:** JavaDoc purpose, generated documentation, IDE tooltips, method and class comment structure.
- **W3.4c:** File access using File, Scanner, FileWriter, append mode, Files, and Paths.
- **W3.4d:** Packages, package naming, imports, fully qualified names, static imports, and command-line compile/run with packages.
- **W3.4e-f:** Access modifiers and JAR files.
- **W3.5a-b:** Code quality, coding standards, recommended approach, IDE support, and AI impact.
- **W3.6a-b:** Developer testing, reasons to test early, and cost of late bug discovery.
- **W3.7a-f:** Test drivers, JUnit, unit testing, test naming, exception testing, stubs, and optional intermediate JUnit awareness.